

Exercise 3

Write a Python program using the `networkx` to create and visualize an undirected graph with the following specifications:

1. Assign **graph-level attributes**:

- "Name" = "My Graph"
- "Description" = "This is my graph"

2. Add 5 nodes with the following attributes:

Node	Age	Gender
A	19	F
B	18	M
C	22	M
D	21	M
E	20	F

3. Add the following edges with weights:

- A — C (weight = 1)
- B — C (weight = 0.5)
- B — D (weight = 0.6)
- C — D (weight = 0.8)
- D — E (weight = 1)

4. Use the following coordinates for positioning nodes:

- A: (2, 4)
- B: (5, 5.5)
- C: (5, 2)
- D: (6, 3)
- E: (8, 3.5)

5. Draw the graph with:

- Node color = red
- Node size = 800
- Font color = yellow
- Font size = 13
- Edge width = 3
- Bold labels

6. Print:

- The graph attributes
- All nodes with their attributes
- All edges with their weights
- Total number of nodes
- Total number of edges
- Degree of each node
- Attributes of node "A"
- Attributes of the edge between "A" and "C"

Solution:

```
import networkx as nx
G = nx.Graph()
G.graph["Name"]="My Graph"
G.graph["Description"]="This is my graph"

G.add_nodes_from([
    ("A",{ "Age":19,"gender":"F"}),
    ("B",{ "Age":18,"gender":"M"}),
    ("C",{ "Age":22,"gender":"M"}),
    ("D",{ "Age":21,"gender":"M"}),
    ("E",{ "Age":20,"gender":"F"}),
])

G.add_edges_from([
    ("A","C",{ "weight":1}),
    ("B","C",{ "weight":0.5}),
    ("B","D",{ "weight":0.6}),
    ("C","D",{ "weight":0.8}),
    ("D","E",{ "weight":1}),
])

pos = {
    "A": (2, 4),
    "B": (5, 5.5),
    "C": (5, 2),
    "D": (6, 3),
    "E": (8, 3.5)
}

nx.draw(G,with_labels=True,font_weight='bold',font_color='yellow',node_color='red',node_size=800,font_size
=13,width=3,pos=pos)

print(G.graph)

for node in G.nodes(data=True):
    print(node)

for edge in G.edges(data=True):
    print(edge)

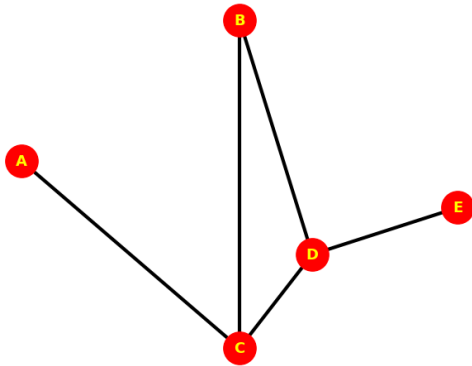
print(G.number_of_nodes())

print(G.number_of_edges())

for node in G.nodes:
    print(f"Degree({node})={G.degree(node)}")

print(G.nodes["A"])

print(G.edges["A","C"])
```



```
{'Name': 'My Graph', 'Description': 'This is my graph'}
```

```
('A', {'Age': 19, 'gender': 'F'})
```

```
('B', {'Age': 18, 'gender': 'M'})
```

```
('C', {'Age': 22, 'gender': 'M'})
```

```
('D', {'Age': 21, 'gender': 'M'})
```

```
('E', {'Age': 20, 'gender': 'F'})
```

```
('A', 'C', {'weight': 1})
```

```
('B', 'C', {'weight': 0.5})
```

```
('B', 'D', {'weight': 0.6})
```

```
('C', 'D', {'weight': 0.8})
```

```
('D', 'E', {'weight': 1})
```

```
5
```

```
5
```

```
Degree(A)=1
```

```
Degree(B)=2
```

```
Degree(C)=3
```

```
Degree(D)=3
```

```
Degree(E)=1
```

```
{'Age': 19, 'gender': 'F'}
```

```
{'weight': 1}
```

Exercise 4

Create an undirected graph with 50 nodes and 100 random edges, and show the following data about the graph:

Number of nodes and edges

Degree of each node

Density, diameter, average shortest path length (for connected components)

Check if the graph is connected.

Visualize graphs using `spring_layout`, `circular_layout`, and `kamada_kawai_layout`.

Plot the degree distribution for the above graph

Solution:

```
import networkx as nx
```

```
import matplotlib.pyplot as plt
```

```
import random
```

```
# Create an undirected graph
```

```
G = nx.Graph()
```

```

# Add 50 nodes
G.add_nodes_from(range(50))

# Add 100 random edges
edges = set()
while len(edges) < 100:
    u = random.randint(0, 49)
    v = random.randint(0, 49)
    if u != v:
        edges.add(tuple(sorted((u, v))))
G.add_edges_from(edges)

# Print number of nodes and edges
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())

# Degree of each node
degrees = dict(G.degree())
print("\nDegree of each node:")
for node, degree in degrees.items():
    print(f"Node {node}: Degree {degree}")

# Graph density
density = nx.density(G)
print("\nDensity of the graph:", density)

# Check if graph is connected
is_connected = nx.is_connected(G)
print("Is the graph connected?", is_connected)

# Compute diameter and average shortest path length for connected components
if is_connected:
    diameter = nx.diameter(G)
    avg_path_length = nx.average_shortest_path_length(G)
else:
    # For each connected component
    diameter = []
    avg_path_length = []
    for component in nx.connected_components(G):
        subgraph = G.subgraph(component)
        diameter.append(nx.diameter(subgraph))
        avg_path_length.append(nx.average_shortest_path_length(subgraph))
    print("Diameters of connected components:", diameter)
    print("Average shortest path lengths of components:", avg_path_length)

# Visualize using spring_layout
plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1)
nx.draw(G, pos=nx.spring_layout(G), with_labels=True, node_color='lightblue')
plt.title("Spring Layout")

# Visualize using circular_layout

```

```

plt.subplot(1, 3, 2)
nx.draw(G, pos=nx.circular_layout(G), with_labels=True, node_color='lightgreen')
plt.title("Circular Layout")

# Visualize using kamada_kawai_layout
plt.subplot(1, 3, 3)
nx.draw(G, pos=nx.kamada_kawai_layout(G), with_labels=True, node_color='lightcoral')
plt.title("Kamada-Kawai Layout")

plt.tight_layout()
plt.show()

# Plot degree distribution
degree_values = list(degrees.values())
plt.figure(figsize=(8, 4))
plt.hist(degree_values, bins=range(min(degree_values), max(degree_values)+2), align='left',
edgecolor='black')
plt.title("Degree Distribution")
plt.xlabel("Degree")
plt.ylabel("Frequency")
plt.grid(True)
plt.show()

```

Number of nodes: 50
Number of edges: 100

Degree of each node:

- Node 0: Degree 4
- Node 1: Degree 3
- Node 2: Degree 2
- Node 3: Degree 3
- Node 4: Degree 2
- Node 5: Degree 6
- Node 6: Degree 3
- Node 7: Degree 2
- Node 8: Degree 2
- Node 9: Degree 3
- Node 10: Degree 4
- Node 11: Degree 6
- Node 12: Degree 3
- Node 13: Degree 5
- Node 14: Degree 3
- Node 15: Degree 7
- Node 16: Degree 6
- Node 17: Degree 2
- Node 18: Degree 1
- Node 19: Degree 1
- Node 20: Degree 3
- Node 21: Degree 2
- Node 22: Degree 5
- Node 23: Degree 5

Node 24: Degree 6
 Node 25: Degree 2
 Node 26: Degree 4
 Node 27: Degree 8
 Node 28: Degree 2
 Node 29: Degree 6
 Node 30: Degree 2
 Node 31: Degree 3
 Node 32: Degree 5
 Node 33: Degree 7
 Node 34: Degree 1
 Node 35: Degree 5
 Node 36: Degree 6
 Node 37: Degree 4
 Node 38: Degree 7
 Node 39: Degree 7
 Node 40: Degree 6
 Node 41: Degree 3
 Node 42: Degree 2
 Node 43: Degree 3
 Node 44: Degree 7
 Node 45: Degree 4
 Node 46: Degree 7
 Node 47: Degree 6
 Node 48: Degree 1
 Node 49: Degree 3

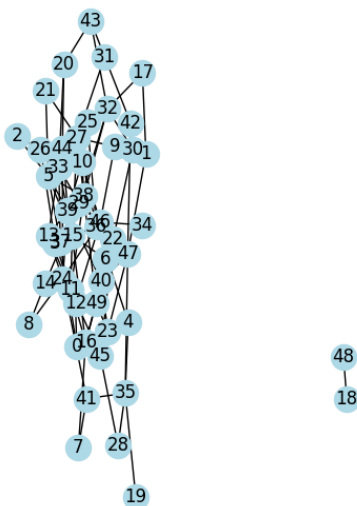
Density of the graph: 0.08163265306122448

Is the graph connected? False

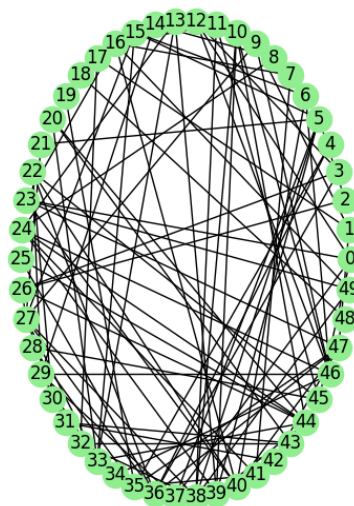
Diameters of connected components: [6, 1]

Average shortest path lengths of components: [2.801418439716312, 1.0]

Spring Layout



Circular Layout



Kamada-Kawai Layout

